Чемпионат «Профессионалы» 2027 · Программные решения для бизнеса

ТОиР-Проф

Система управления техническим обслуживанием и ремонтом

- Оборудование, график ТО, заявки, бригады и склад ЗИП — в одной базе
- PostgreSQL · REST API на .NET 10 · Desktop Avalonia · Web React

Проблема

- График ТО, заявки и склад ЗИП ведутся в разных таблицах
- Невозможно посчитать простой оборудования и долю аварийных
- Не видно, какая бригада чаще задерживает работы

Цель

- База с ограничениями, которые не дают внести неправильные данные
- Один REST API для двух клиентов
- Аналитика: простои, рейтинг бригад, выполнение графика ТО

На экране: две колонки «проблема / цель».

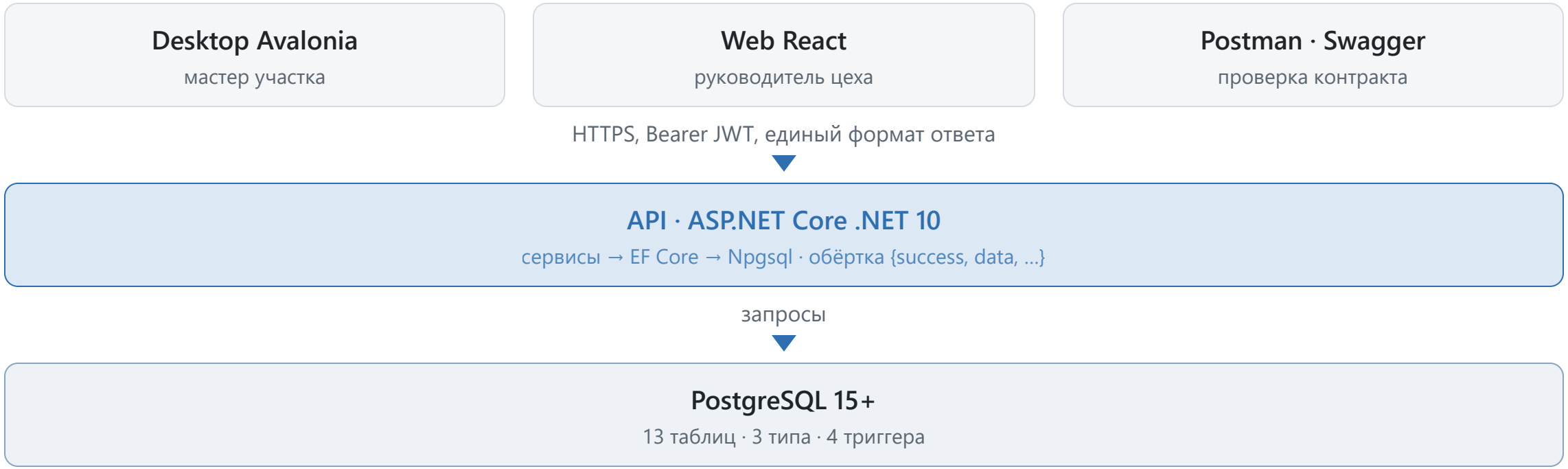
КЛЮЧЕВЫЕ РОЛИ И СЦЕНАРИЙ РАБОТЫ

РОЛЬ	ЧТО ДЕЛАЕТ
Мастер участка	Заводит заявку, назначает бригаду, отмечает выполнение, списывает материалы
Руководитель цеха	Смотрит сводку и аналитику, контролирует простои и рейтинг бригад
Администратор	Ведёт справочники: оборудование, виды ТО, бригады, материалы, пользователи

```
График ТО → заявка на ремонт → назначение бригады → выполнение и списание → аналитика
```

На экране: таблица ролей и схема цикла работы.

АРХИТЕКТУРА РЕШЕНИЯ



- Логика проверок живёт в базе — иначе клиенты посчитали бы по-разному
- Бизнес-правил на клиенте нет: только отображение и отправка данных

МОДЕЛЬ ДАННЫХ

13 таблиц — 9 основных и 4 связующие или служебные

ГРУППА	ТАБЛИЦЫ
Справочники	`roles`, `workshops`, `maintenance_types`, `crews`, `materials`
Учёт	`users`, `equipment`, `maintenance_schedule`, `work_orders`
Связи и служебные	`crew_members`, `work_order_materials`, `operation_log`, `work_order_counters`

- 1НФ — списки вынесены в отдельные таблицы, произвольный текст заменён перечислениями
- 2НФ — связь «сотрудник ↔ бригада» вынесена в `crew_members`
- 3НФ — стоимость списания и простой считает база, а не хранит кто-то лишнюю копию

На экране: `docs/diagrams/class-model.png` — диаграмма классов.

ОГРАНИЧЕНИЯ ЦЕЛОСТНОСТИ

ВИД	ПРИМЕР В СИСТЕМЕ
`UNIQUE`	логин, инвентарный номер, артикул, номер заявки
`CHECK`	хронология дат, критичность 1…5, формат номера `Z-YYYY-XXXX`
Перечисления	статусы оборудования, заявки и приоритет заданы типами базы
`ON DELETE`	график удаляется каскадом, справочники защищены `RESTRICT`
Вычисляемые поля	стоимость = количество × цена, простой считает функция

- Ошибка из базы доходит до клиента понятным текстом с кодом, а не как «500»
- Проверяется скриптом `db/verify.sql`: каждая строка отчёта — отдельная проверка

На экране: DBeaver, вкладка «Ограничения» таблицы `equipment`.

ТРИГГЕРЫ, ФУНКЦИИ И ПРОЦЕДУРА

ОБЪЕКТ	НАЗНАЧЕНИЕ
Триггер 1	Статус оборудования, выведенного из эксплуатации, изменить нельзя
Триггер 2	Проверка дат заявки и пересчёт простоя при каждой записи
Триггеры 3 и 4	Журнал операций: кто и что изменил
Функция 1	`fn_calculate_downtime` — простой в часах между двумя метками времени
Функция 2	`fn_next_maintenance_date` — следующая дата по периодичности вида ТО
Процедура	`proc_workshop_report` — сводный отчёт по цеху за период

На экране: вывод `db/verify.sql`, где каждая проверка даёт строку «OK».

REST API

```
{
  "success": true,
  "data": { "...": "..." },
  "message": "OK",
  "error_code": null
}
```

Коды ответов: 200 · 201 · 400 · 401 · 403 · 404 · 409 · 429 · 500

КОД ОШИБКИ	КОГДА ВОЗВРАЩАЕТСЯ
`STATUS_LOCKED`	попытка вернуть в работу выведенное оборудование
`CREW_BUSY`	бригада уже ведёт другую заявку «В работе»
`NOT_ENOUGH_STOCK`	на складе меньше, чем пытаются списать

На экране: Swagger UI с нажатой кнопкой «Authorize».

DESKTOP-КЛИЕНТ

Мастер работает в восьми окнах: авторизация, дашборд, оборудование, график ТО, заявки, бригады, материалы, отчёты.

- Нативный интерфейс на Avalonia — без встраивания веб-страниц
- Индикатор связи, всплывающие сообщения, модальные окна ошибок
- **Работа без сети:** показываются последние загруженные данные, операции складываются в очередь и уходят на сервер сами, как только связь вернётся
- Локальный журнал: `%LOCALAPPDATA%/ToirProf/logs/client.log`

На экране: окно «Заявки» с выбранной заявкой и историей изменений.

WEB-КЛИЕНТ: СВОДКА И ОБНОВЛЕНИЯ

- Шесть KPI-карточек: заявки, средний простой, доля аварийных, выполнение плана ТО, списание, цехи
- Круговая, столбчатая и линейная диаграммы, топ-5 единиц по простою, последние заявки
- Обновление в реальном времени через Server-Sent Events, а если поток недоступен — резервный опрос раз в минуту с переподключением

Индикатор в шапке:

SSE данные приходят потоком

Опрос поток недоступен, данные забираются раз в 60 секунд

Нет связи сервер недоступен, показаны последние загруженные данные

На экране: дашборд с индикатором `SSE`.

Пять блоков аналитики

№	БЛОК
1	Сводка по заявкам
2	Анализ простоев — топ единиц оборудования
3	Расход материалов
4	Выполнение графика ТО
5	Эффективность бригад с рейтингом

- Выгрузка каждого блока в CSV, PNG и PDF; вся страница печатается
- На сервере отчёт в PDF по временной ссылке, которая живёт 15 секунд
- Данные блоков берутся одним запросом, поэтому цифры не расходятся

На экране: раздел «Аналитика», раскрытые блоки и меню экспорта.

ТЕСТИРОВАНИЕ

ТЕСТ	ЧТО ПРОВЕРЯЕТ
`calculateDowntimeHours`	корректные даты, окончание раньше начала, пустые значения
`validateWorkOrder`	все поля заполнены, нет обязательного, аварийная без причины, неверные плановые даты
`calculateTotalMaterialCost`	несколько материалов, пустой список, нулевое и отрицательное количество

Эталон из задания: 4 × 850 + 2 × 1200 + 10 × 150 = **7 300,00**

- Все 13 проверок проходят; логика вынесена отдельно от интерфейса, поэтому тесты не требуют ни окна, ни базы
- Ограничения базы проверяет скрипт `db/verify.sql`

На экране: вывод `dotnet test` и отчёт `db/verify.sql`.

ДОКУМЕНТАЦИЯ И РАЗВЁРТЫВАНИЕ

- **Инструкция по развёртыванию** — от чистой машины до работающего сервиса: РедОС 8.0, PostgreSQL, API, веб-клиент, Nginx
- **Swagger** — описания операций из XML-комментариев, кнопка «Authorize», примеры запросов
- **Руководства пользователя** — отдельно для мастера и для руководителя
- **Postman-коллекция** — проверка контракта без клиентов
- **UML-диаграммы** — варианты использования, деятельность и классы, исходники в Draw.io

---

На экране: страница `docs/deployment.md` и Swagger UI.

Что сделано

- База данных: 13 таблиц, 3 типа, 30 ограничений, 4 триггера, 3 функции и процедура
- API: 13 контроллеров, единый формат ответа, роли и ограничения доступа
- Desktop-клиент: восемь экранов, работа без сети
- Веб-клиент: дашборд, аналитика, реестры, обновление в реальном времени
- 13 модульных тестов, полная документация, инструкция развёртывания

Дальше

- Уведомления о просроченных работах в мессенджер
- Планирование ТО по ресурсам бригад
- Мобильный клиент для члена бригады

На экране: финальный слайд, переход к вопросам.